ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Tzu-Hsiang Tu**
- **Chinese Name: 涂紫翔**
- **Email: s115033632@m115.nthu.edu.tw**
- **Student ID: 115033632**
- **University: National Tsing Hua University**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1. 594

2. Different categories focus on different topics. If we don't separate them, the meaningful patterns of each category may be averaged out: their support won't reach the minSup, and only common words like "subject" will be left. Also, dividing the data into different groups may improve efficiency, because each dataset is smaller and there are fewer candidate patterns to check.

AI used for this answer: Yes

If yes, how: Grammar polishing and feedback on my own draft.

External references (links): https://en.wikipedia.org/wiki/Association_rule_learning

---

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer:

Dropping 40% of the data means losing a large part of the information our model can learn from or analyze, even though the "text" of those rows is still complete and useful. Furthermore, the missing labels may be related to the category itself (for example, some categories may be unlabeled more often). If we drop them, the distribution may shift and the dataset will become biased. Instead, we can train a classifier on the 60% labeled documents and use it to predict the missing labels, or simply label them as "unknown" so the text can still be used.

AI used for this answer: Yes

If yes, how: Grammar polishing and feedback on my own draft.

External references (links): https://en.wikipedia.org/wiki/Missing_data

https://scikit-learn.org/stable/modules/semi_supervised.html

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1. The proportions are not exactly the same, but they are quite close. The largest difference is about 1.9% (comp.graphics).

2. With simple random sampling, minority categories may be sampled too little or not at all. If we train or analyze on this kind of sample, the result may not represent the whole dataset and may lean toward the majority categories. To avoid this, we can use stratified sampling, which samples each category based on its original proportion. In my test, the stratified sample matched the original proportions to three decimal places.

AI used for this answer: Yes

If yes, how: Grammar polishing and feedback on my own draft.

External references (links): https://en.wikipedia.org/wiki/Stratified_sampling

---

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1. The total number of features increases from 35,482 to 35,788 (+306), because the stop words are kept as features.

2. The sparsity slightly decreases (from 0.996772 to 0.995470), since stop words appear in almost every document, so these new columns contain many non-zero values. However, they are only 306 out of 35,788 columns (less than 1%), so the matrix is still extremely sparse.

AI used for this answer: Yes

If yes, how: Grammar polishing and feedback on my own draft.

External references (links):

https://scikit-learn.org/stable/modules/feature_extraction.html#sparsity

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

---

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

Threshold	**Number of terms kept (keep == True)**
0.0	35482
0.01	5089
0.05	1491

1. As the threshold increases from 0.0 to 0.01 to 0.05, the number of kept features decreases from 35,482 to 5,089 to 1,491. The higher the threshold, the fewer features are kept.

2. Rare words with a very low document frequency are eliminated first, such as names, email accounts, typos, or gibberish (e.g. “ostroff”, “oswald”, “otheres”). Their variance is about $0.000443 \approx 1/2257$, which means they appear in only one document.

3. If a feature’s variance is close to zero, its value is almost the same in every document, so it cannot help distinguish documents or categories and contains little information. Removing these features reduces dimensionality, noise, and computation cost without losing much useful information. However, if the threshold is too high, we may also eliminate informative words.

AI used for this answer: Yes

If yes, how: Grammar polishing and feedback on my own draft.

External references (links):

https://scikit-learn.org/stable/modules/feature_selection.html#removing-features-with-low-variance

---

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

1. For Spearman, the top words are mostly domain-specific terms for graphics, such as “graphics”, “files”, “image”, “3d”, and “program”. For Pearson, there are some domain-related words like “vga”, “video”, and “card”, but there are also many generic words, such as “thanks”, “hi”, and “looking” (common in question posts), and “writes” and “host” (from email headers). In both lists, words like “people”, “writes”, and “god” have negative correlations, which means they appear more often outside comp.graphics.

2. Pearson measures the linear relationship between the raw values, so it is easily affected by outliers. Spearman first transforms the values into ranks and then measures the monotonic relationship, so it is not sensitive to outliers.

3. This happens when a word appears in many comp.graphics documents but only once or twice in most of them, while a few documents use it many times. These outliers lower the Pearson correlation, but the ranks still follow the topic well, so Spearman remains high. For example, “graphics” has a Pearson r of only 0.178, but a Spearman r of 0.521.

AI used for this answer: Yes

If yes, how: Grammar polishing and feedback on my own draft.

External references (links):

https://en.wikipedia.org/wiki/Spearman%27s_rank_correlation_coefficient

https://en.wikipedia.org/wiki/Pearson_correlation_coefficient

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:

When minSup decreases from 9 to 3, the number of patterns increases from 710 to 9,547, and the runtime increases from 0.061 s to 1.41 s. With a lower threshold, many more itemsets pass the threshold, so there are more candidates to check and more patterns to output.

Answer (2) — Top-K k vs MaxFPGrowth minSup:

minSup is an absolute threshold: every pattern with support ≥ minSup is kept, so we don't know how many results we will get in advance. k keeps only the top-k patterns with the highest support, so the number of results is fixed (k=100 took 0.024 s, k=500 took 0.107 s). If we don't know the dataset well, k is easier to tune, while a badly chosen minSup may lead to too many or too few results.

Answer (3) — Maximal vs standard frequent patterns:

A maximal frequent pattern is a frequent pattern that has no frequent superset. Frequent patterns have the downward closure property (every subset of a frequent pattern is also frequent), so the normal result contains many repeated subsets. Maximal patterns keep only the largest ones, and all other frequent patterns can be derived as their subsets, so the result is much smaller.

AI used for this answer: Yes

If yes, how: Grammar polishing and feedback on my own draft.

External references (links): https://en.wikipedia.org/wiki/Apriori_algorithm

---

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

1. A scatter plot can only show 2 or 3 dimensions, but each document in X_counts is a vector with 35,482 dimensions (one per word), so it cannot be placed directly on a 2D or 3D plot.

2. Dimensionality reduction algorithms, such as PCA, t-SNE, or UMAP, which project the data into 2 or 3 dimensions. The functions compute_dimred_2d_for_filters and compute_dimred_3d in helpers/visualization_helpers.py do this.

AI used for this answer: Yes

If yes, how: Grammar polishing and feedback on my own draft.

External references (links): https://en.wikipedia.org/wiki/Dimensionality_reduction

---

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP)** shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1. t-SNE and UMAP show the categories in separate regions, and UMAP even forms some distinct islands (e.g. a small black cluster far from the others). PCA shows heavy

overlap: almost all points are squeezed into one narrow column with all four categories mixed, and only a few outliers are far away.

2. PCA is a linear projection that keeps the directions with the largest variance. Text data is sparse and high-dimensional, so the first 2–3 components explain only a small part of the information, and these directions can be dominated by a few extreme documents (the outliers in the PCA plot) instead of the category differences. t-SNE and UMAP are non-linear and preserve local neighborhoods, so similar documents stay close and form clusters.

3. t-SNE optimizes the local neighbor structure, so the distances between clusters are not preserved, and cluster sizes and densities can also be distorted. The result is also affected by the perplexity and the random state. Therefore, we can say points in the same cluster are similar, but we cannot say one cluster is “closer” to another based on their distance in the plot.

AI used for this answer: Yes

If yes, how: Grammar polishing and feedback on my own draft.

External references (links): https://distill.pub/2016/misread-tsne/

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. 4 binary columns are created, one column for each category.

2. The algorithm may mistake the categories for ordered values (3 > 2 > 1 > 0) and assume they have meaningful distances. For example, it would think category 0 is closer to category 1 (distance 1) than to category 3 (distance 3), although the categories have no such relationship. Linear regression would assume each +1 step has the same effect, and KNN would compute neighbors with these false distances.

3. One-hot binarization makes each category an independent dimension, so there is no order between the vectors.

AI used for this answer: Yes

If yes, how: Grammar polishing and feedback on my own draft.

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelBinarizer.html
https://scikit-learn.org/stable/modules/preprocessing.html#encoding-categorical-features

---